

Evaluación técnica del framework forge

para desarrollo con agentes de IA

Análisis técnico independiente

3 de mayo de 2026
Versión 1.0

Resumen

El presente informe evalúa **forge**, un framework de trabajo para equipos que desarrollan software con agentes de inteligencia artificial. El análisis integra dos perspectivas técnicas independientes —una crítica y una favorable— sobre la misma base de código, commit `d828157`, con el objetivo de producir una evaluación balanceada que permita a equipos de desarrollo tomar una decisión informada sobre la adopción del framework. La conclusión principal es que forge contiene ideas de diseño sólidas implementadas parcialmente, cuya aplicabilidad queda limitada a casos de uso específicos en su estado actual.

Índice

1. Introducción y contexto	2
1.1. El problema que forge intenta resolver	2
1.2. Descripción del framework	2
1.3. Alcance del análisis	2
2. Metodología de análisis	3
3. Hallazgos del análisis crítico	3
3.1. Los adapters para runtimes alternativos no existen	3
3.2. El ecosistema de profiles cubre cuatro stacks	3
3.3. Bug en la función <code>install_agent</code>	3
3.4. El audit recomienda un comando inexistente	4
3.5. El generador de <code>CLAUDE.md</code> desconecta la fuente de verdad	4
3.6. El pre-commit hook muta archivos sin visibilidad del desarrollador	4
4. Hallazgos del análisis favorable	5
4.1. La taxonomía de tres tiers resuelve el problema de los agentes God-object	5
4.2. Las reglas de seguridad son específicas y verificables	5
4.3. El skill <code>new-feature</code> codifica un pipeline de calidad completo	5
4.4. El diseño de skills evita el acoplamiento circular	6
4.5. La auditoría automatizada tiene valor real en CI	6

5. Análisis comparativo	6
5.1. Diseño versus implementación	6
5.2. Comparación con alternativas	6
5.3. Riesgos sistémicos	6
6. Conclusiones y recomendaciones	7
6.1. Conclusión general	7
6.2. Condiciones de adopción recomendable	7
6.3. Recomendaciones para equipos adoptantes	8
6.4. Recomendaciones para el equipo de forge	8
A. Estado de los adapters (mayo 2026)	8
B. Bug completo en install_agent	8
C. Grafo de dependencias de skills	9

1. Introducción y contexto

1.1. El problema que forge intenta resolver

El desarrollo de software asistido por agentes de IA enfrenta un problema de coordinación que se vuelve crítico a escala de equipo. Sin convenciones explícitas, los agentes tienden a:

- Actuar fuera del scope asignado, modificando componentes no relacionados.
- Implementar funcionalidades sin verificación de seguridad (autorización, sanitización de inputs, protección de PII).
- Producir código inconsistente entre sesiones porque sus reglas están en la memoria de cada desarrollador, no en archivos versionados.
- Desactualizarse de las prácticas del equipo a medida que el proyecto evoluciona.

forge propone una respuesta estructural: codificar las convenciones del equipo en archivos que los agentes lean, versionados en un repositorio central, y propagados a los proyectos mediante scripts determinísticos.

1.2. Descripción del framework

forge se integra a un proyecto mediante `git submodule`:

```
1 git submodule add https://github.com/tu-org/forge .agentic
2 python3 .agentic/scripts/forge-init.py --tool claude-code
```

Listing 1: Instalación básica de forge

Una vez instalado, el desarrollador describe su proyecto en un `project.yaml` y ejecuta `forge-init.py` para generar la configuración de los agentes. La estructura del repositorio incluye:

- `core/agents/`: agentes genéricos (orchestrator, backend-engineer, security-auditor, compliance-reviewer, test-engineer, frontend-engineer, mobile-engineer).
- `profiles/<stack>/agents/`: agentes especializados por stack tecnológico.
- `core/skills/`: skills reutilizables (new-feature, security-audit, db-migrate, local2prod, browser-test).
- `adapters/`: scripts para generar configuración en distintos runtimes (Claude Code, OpenCode, Kiro).
- `scripts/`: herramientas de gestión (forge-init.py, forge-audit.py).
- `hooks/`: hook de pre-commit para estadísticas de tokens.

1.3. Alcance del análisis

El análisis cubre el commit `d828157` del repositorio `forge`. Los archivos analizados incluyen: `README.md`, `templates/project.yaml.tpl`, `scripts/forge-init.py`, `scripts/forge-audit.py`, `adapters/claude-code/generate-claude-md.py`, siete agentes del core, `core/skills/README.md`, cuatro skills, cuatro profiles, `hooks/pre-commit`, `docs/agent-standard.md`, y `core/workflows/sdd.md`.

2. Metodología de análisis

El análisis se realizó mediante lectura directa del código fuente, sin ejecutar el framework en un entorno de prueba. Se adoptó un enfoque adversarial estructurado: dos analistas independientes revisaron el mismo repositorio con instrucciones opuestas —uno buscando argumentos en contra, otro en favor— y produjeron informes separados. El presente documento integra ambos informes con el objetivo de producir una evaluación libre de sesgos de confirmación.

La evaluación de cada componente siguió tres preguntas:

1. ¿Qué problema declara resolver este componente?
2. ¿El código implementa efectivamente esa solución?
3. ¿Qué condiciones deben cumplirse para que la implementación aporte valor neto positivo?

Las evidencias citadas son fragmentos directos del código fuente del repositorio, no inferencias sobre la intención de los autores.

3. Hallazgos del análisis crítico

3.1. Los adapters para runtimes alternativos no existen

El README de forge promete compatibilidad con Claude Code, OpenCode, Codex y otros runtimes. La inspección del código revela una brecha significativa:

- `adapters/claude-code/`: funcional.
- `adapters/opencode/`: directorio vacío. Cero archivos.
- `adapters/kiro/`: directorio vacío. Cero archivos.

La función `init_kiro()` en `forge-init.py` existe y genera dos archivos `.md` con contenido genérico. La comparación de complejidad es ilustrativa: `init_claude_code` tiene aproximadamente 130 líneas de lógica; `init_kiro` tiene aproximadamente 30 y produce documentos de placeholder sin aprovechar ningún concepto del framework.

3.2. El ecosistema de profiles cubre cuatro stacks

El sistema de profiles es la parte conceptualmente más valiosa del framework. La realidad del inventario en el commit analizado se muestra en la Tabla 1.

3.3. Bug en la función `install_agent`

En `scripts/forge-init.py`, la función de instalación de agentes contiene un defecto lógico:

Cuadro 1: Profiles implementados en forge (mayo 2026)

Profile	Agente incluido
expo	mobile-engineer.md
hono-drizzle	api-engineer.md
nextjs-admin	admin-engineer.md
playwright-crawler	scanner-engineer.md
<i>Stacks sin profile: Django, Express, Laravel, NestJS, Nuxt</i>	

```

1 def install_agent(src: Path, dst: Path, name: str,
2   source_label: str) -> str:
3     if not src.exists():
4       return "MISS"
5     if dst.exists() and not FORCE:
6       return "KEEP"
7     shutil.copy2(src, dst)
8     # Bug: dst.exists() siempre True despues de copy2
9     return "UPDATE" if dst.exists() else "OK"

```

Listing 2: Bug en install_agent: status .OK"nunca retorna

La evaluación `dst.exists()` ocurre después de que `shutil.copy2` ya copió el archivo. En consecuencia, `dst.exists()` siempre es `True` en ese punto, y el status `.OK` nunca se devuelve.

3.4. El audit recomienda un comando inexistente

`forge-audit.py` genera mensajes de corrección de la forma:

```

1 "fix": f"forge-init.py --tool claude-code --force
2   --only={agent['name']}"
3 # El flag --only no existe en forge-init.py

```

Listing 3: Comando de fix con flag inexistente

Un desarrollador que siga la recomendación del audit ejecutará un comando que falla.

3.5. El generador de CLAUDE.md desconecta la fuente de verdad

El `project.yaml` contiene una sección `sprint.phases`. El `generate-claude-md.py` genera una sección "Phases activas y estado con contenido fijo, no conectado con los datos del YAML. El usuario debe actualizar manualmente la sección más dinámica del documento más importante que los agentes leerán.

3.6. El pre-commit hook muta archivos sin visibilidad del desarrollador

El hook ejecuta `token-stats.py`, modifica `docs/progress.html`, y hace `git add` sobre ese archivo antes de completar el commit. El archivo entra al commit sin haber sido

inspeccionado. Los errores del script se suprimen con `|| true`, haciendo que los fallos sean silenciosos.

4. Hallazgos del análisis favorable

4.1. La taxonomía de tres tiers resuelve el problema de los agentes God-object

El `docs/agent-standard.md` define tres niveles con criterios de clasificación claros y aplicables:

- **Tier 1 (Universal):** definidos por tipo de output, no por tecnología. El `orchestrator`, `security-auditor`, `compliance-reviewer` y `test-engineer` son genuinamente agnósticos al stack.
- **Tier 2 (Profile):** mismo rol que Tier 1 con instrucciones específicas al stack. El `api-engineer` de `hono-drizzle` sabe que las migraciones reversibles requieren un `down` manual y que el runtime en dev es Bun pero en producción es Node 22 LTS.
- **Tier 3 (Dominio):** agentes que conocen conceptos del negocio. Viven en el proyecto, no en forge. El framework los reconoce y los verifica en la auditoría sin prescribirlos.

4.2. Las reglas de seguridad son específicas y verificables

Los agentes de forge no mencionan seguridad como aspiración: la codifican como reglas específicas. El `backend-engineer` incluye instrucciones no ambiguas:

```
1 - Usa parametros preparados siempre -- nunca concatenar inputs
   en queries SQL
2 - Verifica autenticacion Y autorizacion en cada endpoint
3 - No loguear PII. Solo IDs hash o indicadores
4 - No exponer detalles tecnicos de errores al cliente en
   produccion
```

Listing 4: Reglas de seguridad en `backend-engineer.md`

El `api-engineer` de `hono-drizzle` agrega restricciones específicas al stack:

```
1 - Logs de auditoria son append-only. NUNCA UPDATE ni DELETE
2 - Multi-tenancy: toda query filtra por tenant_id. Sin
   excepciones
3 - Migraciones reversibles: toda migracion tiene down
```

Listing 5: Reglas específicas de stack en `api-engineer.md`

4.3. El skill `new-feature` codifica un pipeline de calidad completo

El `skill core/skills/new-feature/SKILL.md` define seis fases secuenciales con condiciones de corte: (1) verificar spec aprobada, (2) leer documentación del área, (3) evaluar si justifica un team de agentes, (4) checklist de seguridad, (5) orden de implementación

(schema → tipos → backend → frontend → build check), y (6) post-implementación completa.

Los agentes no pueden *terminar* una tarea sin haber pasado por security audit y haber verificado el deploy.

4.4. El diseño de skills evita el acoplamiento circular

El README.md de skills documenta explícitamente el grafo de dependencias:

```
1 new-feature orquesta:  
2   spec, phase-kickoff, security-audit, db-migrate,  
3   browser-test (opt), local2prod  
4 local2prod invoca: obsidian-sync (opt)  
5 security-audit: standalone  
6 db-migrate: standalone
```

Listing 6: Grafo de dependencias de skills

Las dependencias opcionales se resuelven por configuración en `project.yaml`. Si `browser-test` no está en `skills.active`, `new-feature` lo saltea silenciosamente sin romper el pipeline.

4.5. La auditoría automatizada tiene valor real en CI

`forge-audit.py` acepta `--json` para integración en CI. Detecta: frontmatter incompleto en agentes, modelo incorrecto (Sonnet en lugar de Opus para compliance y security), agentes desactualizados respecto al core, agentes huérfanos no declarados en `project.yaml`, y oportunidades de activar perfiles o skills disponibles. Esta capacidad transforma los estándares de documentación aspiracional a reglas ejecutables.

5. Análisis comparativo

5.1. Diseño versus implementación

El análisis revela una brecha consistente entre la calidad del diseño conceptual y el estado de la implementación. La Tabla 2 resume los hallazgos por dimensión.

5.2. Comparación con alternativas

La alternativa más común es escribir agentes directamente en `.claude/agents/` sin framework: costo de 30 minutos para un `CLAUDE.md` inicial, control total, sin dependencias. `forge` agrega valor cuando el equipo necesita: (a) propagación consistente de reglas de seguridad entre proyectos, (b) agentes especializados por stack mantenidos centralmente, (c) auditoría en CI, o (d) compliance con poder de veto.

5.3. Riesgos sistémicos

Los riesgos más significativos no son los bugs individuales sino los patrones sistémicos:

- **Actualización destructiva:** el mecanismo `--force` sobrescribe personalizaciones sin merge. Con el tiempo, los equipos dejan de actualizar los agentes para proteger sus personalizaciones.

Cuadro 2: Análisis comparativo por dimensión

Dimensión	A favor	En contra
Arquitectura	Taxonomía 3 tiers coherente	Solo 4 profiles implementados
Fuente de verdad	project.yaml centraliza todo	Fases del sprint no se propagan
Seguridad	Reglas específicas y verificables	Compliance sin acceso a texto legal
Multi-runtime	Diseño limpio de separación	OpenCode y Kiro: vacíos
Auditoría	Output JSON integrable en CI	Flag <code>-only</code> del fix no existe
Actualización	Propagación desde repo central	<code>-force</code> destruye customizaciones
Adopción	Setup en un solo comando	Sin comando de teardown

- **Dependencia sin escape:** no existe comando de teardown. La salida requiere limpieza manual que puede ser omitida.
- **Falsa seguridad en compliance:** el `compliance-reviewer` es un primer filtro valioso, pero el framework no advierte que opera sin acceso a texto legal oficial.

6. Conclusiones y recomendaciones

6.1. Conclusión general

forge es un framework con arquitectura conceptual correcta e implementación en estado temprano. La taxonomía de tres tiers resuelve un problema real de forma elegante. Las reglas de seguridad codificadas en los agentes son específicas, verificables y no se degradan entre sesiones. El pipeline del skill `new-feature` define una barra de calidad ejecutable. Sin embargo, los adapters para runtimes alternativos son directorios vacíos, el ecosistema de profiles cubre cuatro stacks, y el tooling de auditoría tiene bugs que afectan directamente la experiencia del usuario.

6.2. Condiciones de adopción recomendable

Se recomienda adoptar forge cuando se cumplen simultáneamente:

1. Equipo de 3 a 8 personas usando agentes de IA activamente en el loop de desarrollo.
2. Stack cubierto por alguno de los cuatro profiles existentes.
3. Claude Code como runtime principal.
4. Necesidades de compliance reales (GDPR, Ley 21.719, LGPD).
5. Tolerancia para customizaciones mínimas de agentes.

6.3. Recomendaciones para equipos adoptantes

- Documentar la política de uso de `--force`: cuándo está permitido sobrescribir agentes customizados.
- No confiar en los mensajes de fix del audit para comandos con el flag `--only`: ese flag no existe en `forge-init.py`.
- Verificar manualmente la sección de fases del sprint en `CLAUDE.md` después de cada regeneración.
- Tratar el `compliance-reviewer` como primer filtro, no como revisión legal suficiente.

6.4. Recomendaciones para el equipo de forge

- Implementar los adapters para OpenCode y Kiro o remover su mención del README hasta que estén disponibles.
- Corregir el flag `--only` en los mensajes del audit o implementar el flag en `forge-init.py`.
- Conectar la sección `sprint.phases` de `project.yaml` con la generación de `CLAUDE.md`.
- Agregar un comando de teardown documentado.
- Implementar un mecanismo de merge para actualizaciones de agentes que no destruya customizaciones.

A. Estado de los adapters (mayo 2026)

```
1 adapters/  
2 |-- claude-code/  
3 |   |-- generate-claude-md.py    <- implementado  
4 |   +-- commands/wiki/          <- implementado  
5 |-- opencode/                   <- directorio vacio  
6 +-- kiro/                       <- directorio vacio
```

Listing 7: Estado del directorio adapters/

B. Bug completo en `install_agent`

```
1 def install_agent(src: Path, dst: Path, name: str,  
   source_label: str) -> str:  
2     if not src.exists():  
3         return "MISS"  
4     if dst.exists() and not FORCE:  
5         return "KEEP"  
6     shutil.copy2(src, dst)  
7     # Bug: dst.exists() siempre es True despues de copy2  
8     # El status "OK" para nueva instalacion nunca se retorna
```

```
9 return "UPDATE" if dst.exists() else "OK"
```

Listing 8: Función `install_agent` completa con el bug

C. Grafo de dependencias de skills

```
1 new-feature orquesta:  
2   spec, phase-kickoff, security-audit, db-migrate,  
3   browser-test (opt), local2prod  
4  
5 local2prod invoca:  
6   obsidian-sync (opt)  
7  
8 security-audit: standalone  
9 db-migrate: standalone
```

Listing 9: Grafo de dependencias documentado en `core/skills/README.md`

*Informe generado el 3 de mayo de 2026. Base de análisis: commit **d828157** del repositorio *forge*.*